

1. Linux Fundamentals



1. Linux File System

/	Root directory
/bin	Essential command binaries
/etc	Configuration files
/home	User home directories
/var	Variable files (logs, databases)
/tmp	Temporary files
/usr	User utilities and applications
/opt	Optional add-on applications
/boot	Boot loader files
/dev	Device files



2. Basic Commands

ls	List files and directories
cd	Change directory
pwd	Print working directory
mkdir	Make directory
rm	Remove file or directory
cp	Copy files or directories
mv	Move or rename files
cat	Display file content
less	View file content page by page
head	Show first lines of a file
tail	Show last lines of a file
man	Show manual for command



3. File Permissions

	- r w x	r - x	r - -
	User	Group	Others
r	Read (4)		
w	Write (2)		
x	Execute (1)		
-	No Permission (0)		

Examples:

```
chmod 755 file.sh
chmod 644 file.txt
chown user:group file.txt
```



4. Users & Groups

whoami	Show current user
id	Show user & group info
adduser user	Add a new user
userdel user	Delete a user
passwd user	Set password for user
groupadd group	Add a new group
usermod -aG group user	Add user to group



5. Processes

ps aux	Show all running processes
top	Real-time process monitor
htop	Interactive process viewer
kill PID	Kill process by PID
killall name	Kill by process name
jobs	Show background jobs
bg %n	Resume job in background
fg %n	Bring job to foreground



6. Services (systemctl)

systemctl start service	Start service
systemctl stop service	Stop service
systemctl restart service	Restart service
systemctl status service	Service status
systemctl enable service	Enable at boot
systemctl disable service	Disable at boot



7. Package Managers

Debian/Ubuntu (apt)

```
sudo apt update
sudo apt upgrade
sudo apt install package
sudo apt remove package
```

RHEL/CentOS (yum/dnf)

```
sudo yum update
sudo yum install package
sudo yum remove package
sudo dnf install package
```



8. Networking Basics

ip a	Show IP address
ip r	Show routing table
ping host	Check connectivity
traceroute host	Trace network path
ss -tuln	List open ports
netstat -tuln	(Legacy) List open ports
hostname	Show hostname
hostname -I	Show IP address
nmcli device status	Network device status



9. SSH

ssh user@host	Connect to remote host
ssh -p port user@host	Connect using port
scp file user@host:/path	Copy file to remote
scp user@host:/path file	Copy file from remote
ssh-keygen	Generate SSH key
ssh-copy-id user@host	Copy public key



10. Bash Basics

Variables

```
name="DevOps"
echo $name
```

Special Variables

\$0	Script name
\$1	First argument
\$@	All arguments
\$?	Last command exit status

Useful Operators

==	Equal
!=	Not equal
-eq	Equal (numbers)
-ne	Not equal (numbers)
-gt	Greater than
-lt	Less than

I/O Redirection

command > file	Overwrite
command >> file	Append
command < file	Input from file
command 2> error.log	Error output
command > out.log 2>&1	Both to file

Pipes

```
command1 | command2
command1 | grep "text"
command1 | wc -l
```



PYTHON LIFE

2. Shell Scripting

>_

X= 1. Variables

No spaces around =

```
name="DevOps"
age=25

echo $name # DevOps
echo $age  # 25
```

2. User Input

```
read var

echo "Enter your name: "

read name

echo "Hello, $name"
```

+ - 3. Operators

```
a=10
b=20

sum=$((a+b)) # 30
diff=$((b-a)) # 10
mul=$((a*b)) # 200
div=$((b/a)) # 2
```

Comparison

-eq	Equal
-ne	Not equal
-gt	Greater than
-lt	Less than
-ge	Greater or equal
-le	Less or equal

Logical

-a	AND
-o	OR
!	NOT

? 4. Conditions

```
if [ $age -ge 18 ]; then
    echo "Adult"
elif [ $age -lt 18 ]; then
    echo "Minor"
else
    echo "Invalid"
fi
```

5. Loops

```
For Loops
for i in 1 2 3 4 5; do
    echo $i
done

While Loop
i=1
while [ $i -le 5 ]; do
    echo $i
    i=$((i+1))
done

Until Loop
i=1
until [ $i -gt 5 ]; do
    echo $i
    i=$((i+1))
done
```

f() 6. Functions

```
greet() {
    echo "Hello, $1"
}

greet "DevOps"
```

7. Arrays

```
nums=(10 20 30 40)

echo ${nums[0]} # 10
echo ${nums[@]} # 10 20 30 40
echo ${#nums[@]} # 4
```

"abc" 8. String Operations

```
str="DevOpsTools"

echo ${str:0:3} # Dev
echo ${str:3}   # OpsTools
echo ${#str}    # 11
echo ${str//o/O} # DevOpsT00ls
```

9. File Handling

```
file="test.txt"
echo "Hello" > $file # Write
cat $file           # Read
echo "World" >> $file # Append
rm $file            # Delete

if [ -f $file ]; then
    echo "File exists"
fi
```

</> 10. Automation Scripts

```
#!/bin/bash
# Simple Backup Script
src_dir="/home/user/data"
backup_dir="/home/user/backup"
date=$(date +%F)

mkdir -p $backup_dir
cp -r $src_dir $backup_dir/backup_$date
echo "Backup completed on $date"
```

💡 Tips

- ✓ First line should be shebang `#!/bin/bash`
- ✓ Use meaningful variable names
- ✓ Use quotes to avoid word splitting
- ✓ Use comments for better readability
- ✓ Test your script before automation
- ✓ Make script executable: `chmod +x script.sh`

🐍 PYTHON LIFE

3. Git & GitHub



1. Git Installation

```
# Linux (Ubuntu)
sudo apt update
sudo apt install git

# macOS
brew install git

# Windows
Download from git-scm.com
```

2. Initialize Repository

```
git init
```

Initialize a new Git repository in the current directory.

3. Clone Repository

```
git clone <repo_url>
```

Clone an existing repository from remote to local.

4. Check Status

```
git status
```

Show the status of changes as modified, staged, or untracked.

5. Add Changes

```
git add <file>
git add .
```

Add a specific file or all changes to the staging area.

6. Commit Changes

```
git commit -m "message"
```

Commit staged changes with a descriptive message.

7. Push Changes

```
git push origin <branch>
```

Push committed changes to the remote repository.

8. Pull Changes

```
git pull origin <branch>
```

Fetch and merge changes from the remote repository.

9. Branching

```
git branch          # List branches
git branch <name>    # Create branch
git checkout <name>  # Switch branch
```

10. Merge

```
git checkout main
git merge <branch>
```

Merge changes from <branch> into the current branch.

11. Rebse

```
git checkout <branch>
git rebase main
```

Reapply commits of <branch> on top of another base branch.

12. Tags

```
git tag <tag_name>    # Create tag
git tag -l             # List tags
git push origin <tag_name> # Push tag
```

13. GitHub Workflow



>_ Common Useful Commands

<code>git log</code>	Show commit history
<code>git log --oneline</code>	Show short log
<code>git diff</code>	Show changes
<code>git diff --staged</code>	Show staged changes
<code>git reset <file></code>	Unstage a file
<code>git reset --hard HEAD</code>	Discard all local changes

<code>git stash</code>	Stash changes
<code>git stash pop</code>	Apply stashed changes
<code>git remote -v</code>	Show remotes
<code>git remote add origin <url></code>	Add remote
<code>git fetch</code>	Fetch from remote
<code>git checkout -b <branch></code>	Create & switch branch

.gitignore Example

<code># Ignore logs</code>	<code># Ignore OS files</code>
<code>*.log</code>	<code>.DS_Store</code>
<code># Ignore node modules</code>	<code>Thumbs.db</code>
<code>node_modules/</code>	<code># Ignore build files</code>
<code># Ignore environment files</code>	<code>dist/</code>
<code>.env</code>	<code>build/</code>

★ Best Practices

- ✓ Commit small & meaningful changes.
- ✓ Write clear commit messages.
- ✓ Pull latest changes before pushing.
- ✓ Use branches for new features or fixes.
- ✓ Review code before merging.
- ✓ Keep your repository clean and organized.



4. Networking



1. OSI Model (7 Layers)

7	Application	Network services to applications
6	Presentation	Data representation, encryption, compression
5	Session	Session management, start/stop sessions
4	Transport	End-to-end connection, reliability (TCP/UDP)
3	Network	Logical addressing, routing (IP)
2	Data Link	Framing, MAC addressing, error detection
1	Physical	Transmission of raw bits over physical media

2. TCP / IP Model (4 Layers)

Application (Layer 4)	HTTP, FTP, SMTP, DNS, DHCP
Transport (Layer 3)	TCP, UDP
Internet (Layer 2)	IP, ICMP, ARP
Network Access (Layer 1)	Ethernet, Wi-Fi, MAC, Physical

3. IP Address

• IPv4 Address Format

192	.	168	.	1	.	1
Network ID				Host ID		

• IPv4 is 32-bit (4 Octets)

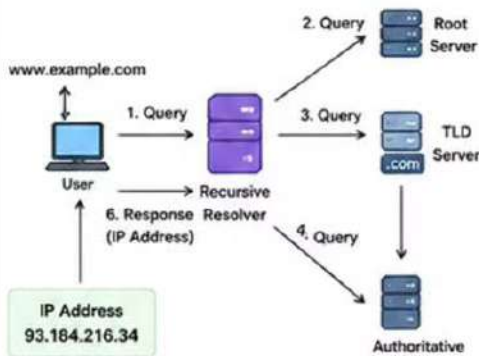
• Subnet Mask example:

255 . 255 . 255 . 0 (/24)

• Private IP Ranges

10.0.0.0 – 10.255.255.255	(Class A)
172.16.0.0 – 172.31.255.255	(Class B)
192.168.0.0 – 192.168.255.255	(Class C)

4. DNS (Domain Name System)



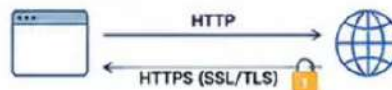
5. HTTP / HTTPS

HTTP (Port 80)

- HyperText Transfer Protocol
- Unencrypted
- Used for web communication

HTTPS (Port 443)

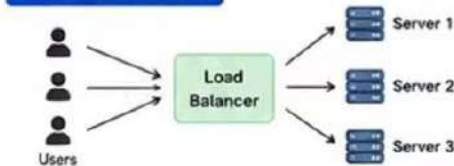
- HTTP Secure
- Encrypted using SSL/TLS
- Secure data transmission



6. Ports

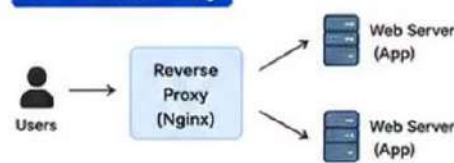
Port	Protocol	Service
21	TCP	FTP
22	TCP	SSH
23	TCP	Telnet
25	TCP	SMTP
53	UDP/TCP	DNS
80	TCP	HTTP
110	TCP	POP3
143	TCP	IMAP
443	TCP	HTTPS
3306	TCP	MySQL

7. Load Balancer



- Distributes incoming traffic across multiple servers
- Improves performance and availability
- Examples: Nginx, HAProxy, AWS ELB

8. Reverse Proxy



- Sits in front of web servers
- Hides real server IPs
- Handles SSL termination, caching, load balancing

9. SSL / TLS



- ✓ Encrypts data between client and server
- ✓ Ensures data integrity
- ✓ Authenticates the server
- ✓ Uses certificates
- ✓ Ports: 443 (HTTPS)

10. Firewalls



- Monitors and filters incoming and outgoing network traffic
- Based on predefined security rules
- Types: Network Firewall, Host Firewall
- Examples: iptables, firewalld, UFW, Cloud Security Groups

11. Common Networking Commands

<code>ping <host></code>	Check connectivity
<code>tracert <host></code>	Trace the route
<code>ip a</code>	Show IP address
<code>ip r</code>	Show routing table
<code>netstat -tln</code>	List open ports
<code>ss -tln</code>	Another way to list ports

<code>nslookup <domain></code>	DNS lookup
<code>dig <domain></code>	DNS query
<code>curl <url></code>	Test HTTP/HTTPS
<code>telnet <host> <port></code>	Test port connectivity
<code>whois <domain></code>	Domain info



Quick Reference

IP Address: Unique identifier for a device on a network

MAC Address: Unique hardware address (Layer 2)

Gateway: Entry point to another network

Subnet Mask: Defines network and host portion

DNS: Converts domain names to IP addresses

5. Programming Basics



1. Python Basics

```
# Variables
name = "DevOps"
age = 25

# Data Types
x = 10          # int
y = 3.14        # float
z = "Hello"     # str
is_active = True # bool

# Output
print("Hello,", name)

# Input
user = input("Enter name: ")
```

2. Functions

```
def greet(name):
    return f"Hello, {name}!"

# Function Call
message = greet("DevOps")
print(message)
```

Types of Functions

- Built-in Functions: print(), len(), type()
- User-defined Functions
- Lambda (Anonymous) Functions

3. OOP (Object-Oriented Programming)

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        return f"Hi, I am {self.name}"

p = Person("DevOps", 25)
print(p.introduce())
```

OOP Concepts

- Class
- Object
- Inheritance
- Encapsulation
- Polymorphism
- Abstraction

4. JSON

```
{
  "name": "DevOps",
  "age": 25,
  "skills": ["Linux", "AWS", "Docker"],
  "is_active": true
}
```

Python (JSON)

```
import json
data = {"name": "DevOps", "age": 25}
json_str = json.dumps(data)
print(json_str)

data_back = json.loads(json_str)
print(data_back["name"])
```

5. YAML

```
name: DevOps
age: 25
skills:
  - Linux
  - AWS
  - Docker
is_active: true
```

Python (YAML)

```
import yaml
data = yaml.safe_load(open("data.yml"))
print(data["name"])
```

6. REST APIs

REST (Representational State Transfer)

- Uses HTTP methods
- Stateless communication
- Data in JSON or XML

Method	Purpose
GET	Retrieve data
POST	Create new data
PUT	Update existing data
DELETE	Delete data

7. Error Handling

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
except Exception as e:
    print("Error:", e)
else:
    print("No error")
finally:
    print("Execution completed")
```

8. Virtual Environments

```
# Create
python -m venv myenv

# Activate
# Windows
myenv\Scripts\activate
# Linux / macOS
source myenv/bin/activate

# Deactivate
deactivate
```

9. Package Managers

```
pip (Python)
pip install package_name
pip uninstall package_name
pip list
pip freeze > requirements.txt
```

Other Tools

```
conda (Anaconda)
npm (Node.js)
apt / yum (System Packages)
```

10. API Requests

Using requests Library

```
import requests
url = "https://api.example.com/data"
response = requests.get(url)

if response.status_code == 200:
    data = response.json()
    print(data)
else:
    print("Error:", response.status_code)
```

Status Codes

```
200 OK
201 Created
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
500 Server Error
```

Common Data Structures

```
List
items = [1, 2, 3]
items.append(4)
items[0]

Tuple
t = (1, 2, 3)
t[0]

Dictionary
d = {"key": "value"}
d["key"]

Set
s = {1, 2, 3}
s.add(4)
```

Common Operators

```
+ Addition
- Subtraction
* Multiplication
/ Division
% Modulus
** Exponent

== Equal
!= Not Equal
> Greater Than
< Less Than
>= Greater or Equal
<= Less or Equal

and Logical AND
or Logical OR
not Logical NOT
```

Helpful Built-in Functions

```
print() Output
len() Length
type() Data Type
int() Integer
float() Float
str() String

list() List
dict() Dictionary
set() Set
sorted() Sort
sum() Sum
max() Maximum
min() Minimum
```

Best Practices

Write clean and readable code

Use meaningful variable names

Add comments where needed

Handle errors gracefully

Follow DRY (Don't Repeat Yourself)

Test your code regularly

Keep your code secure

PYTHON LIFE

6. Docker



1. Images

Blueprints for containers.



Image



2. Containers

Running instances of images.



Container



3. Dockerfile

Instructions to build an image.



Dockerfile



4. Docker Hub

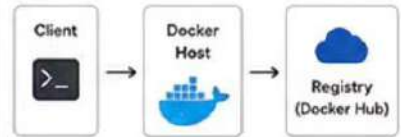
Registry to store and share images.



Docker Hub



5. Architecture



6. Basic Docker Commands

<code>docker --version</code>	Check Docker version	<code>docker images</code>	List all images	<code>docker stop <id></code>	Stop a running container
<code>docker info</code>	Show system-wide info	<code>docker rmi <image></code>	Remove an image	<code>docker restart <id></code>	Restart a container
<code>docker login</code>	Login to Docker Hub	<code>docker ps</code>	List running containers	<code>docker rm <id></code>	Remove a container
<code>docker logout</code>	Logout from Docker Hub	<code>docker ps -a</code>	List all containers	<code>docker logs <id></code>	View container logs
<code>docker search <image></code>	Search images	<code>docker run <image></code>	Run a container	<code>docker exec -it <id> bash</code>	Access container shell
<code>docker pull <image></code>	Download an image	<code>docker start <id></code>	Start a stopped container	<code>docker inspect <id></code>	Inspect container details



7. Dockerfile (Example)

```
# Use official Python image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install -r requirements.txt

# Copy application code
COPY . .

# Expose port
EXPOSE 8000

# Run application
CMD ["python", "app.py"]
```



8. Build & Run

Build an image

```
docker build -t myapp:1.0 .
```

Run a container

```
docker run -d -p 8000:8000 \
--name mycontainer myapp:1.0
```

Run with options

```
docker run -it --rm \
-v /host/data:/app/data \
--name mycontainer myapp:1.0
```



9. Docker Compose

docker-compose.yml (Example)

```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - ./app
    depends_on:
      - db
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - dbdata:/var/lib/postgresql/data
volumes:
  dbdata:
```

Common Commands

```
docker-compose up
Start services

docker-compose up -d
Start in detached mode

docker-compose down
Stop and remove services

docker-compose ps
List services

docker-compose logs
View logs

docker-compose build
Build services
```



10. Volumes

- Persist data outside containers
- Share data between containers
- Types: Named Volumes, Bind Mounts

Commands:

<code>docker volume ls</code>	List volumes
<code>docker volume create myvol</code>	Create volume
<code>docker run -v myvol:/app/data myimage</code>	Use volume
<code>docker volume rm myvol</code>	Remove volume



11. Networks

- Enable communication between containers
- Types: Bridge (default), Host, None, Overlay

Commands:

<code>docker network ls</code>	List networks
<code>docker network create mynet</code>	Create network
<code>docker run --network=mynet myimage</code>	Use network
<code>docker network rm mynet</code>	Remove network



12. Registry (Docker Hub)

- Store and share container images
- Public and Private repositories

Commands:

<code>docker tag myimage username/myimage:1.0</code>	Tag image
<code>docker push username/myimage:1.0</code>	Push image
<code>docker pull username/myimage:1.0</code>	Pull image



13. Best Practices



Use official base images



Keep images small



Don't run as root user



Remove unused images & containers



Use .dockerignore to exclude files



Scan images for vulnerabilities



Use specific tags instead of latest



PYTHON LIFE

7. Kubernetes



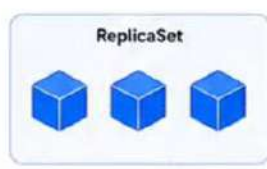
1. Pods

Smallest deployable units in Kubernetes.



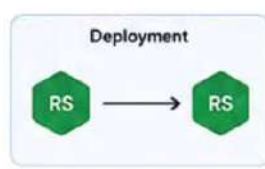
2. ReplicaSets

Ensures the specified number of pod replicas are running.



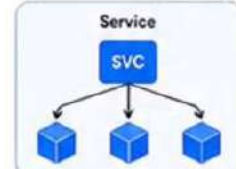
3. Deployments

Manages ReplicaSets and provides declarative updates.



4. Services

Exposes a set of pods as a network service.



5. ConfigMaps

Stores non-confidential configuration data.

```
app.properties:
key1: value1
key2: value2
```



6. Secrets

Stores confidential data like passwords, tokens.

```
username: admin
password: *****
```



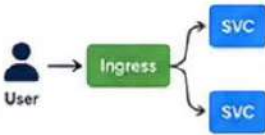
7. Namespaces

Provides a way to divide cluster resources.



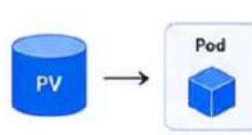
8. Ingress

Manages external access to services (HTTP/HTTPS).



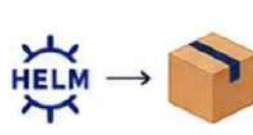
9. Persistent Volumes

Provides persistent storage independent of pod lifecycle.

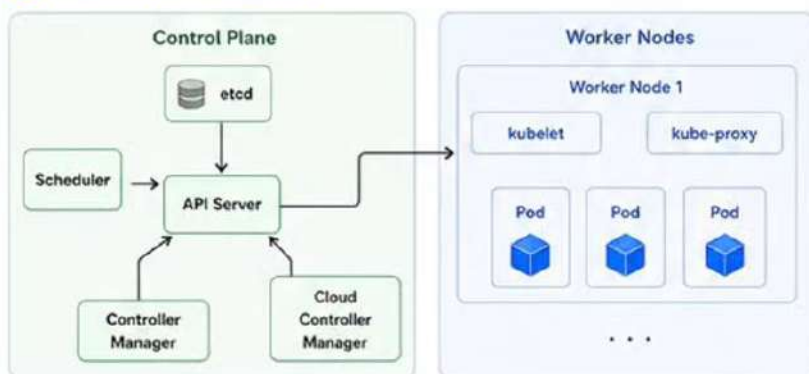


10. Helm

Package manager for Kubernetes applications.



Kubernetes Architecture



Common kubectl Commands

<code>kubectl get pods</code>	List all pods
<code>kubectl get pods -o wide</code>	List pods with more details
<code>kubectl get nodes</code>	List all nodes
<code>kubectl get svc</code>	List all services
<code>kubectl get deployments</code>	List all deployments
<code>kubectl describe <resource> <name></code>	Show details of a resource
<code>kubectl logs <pod-name></code>	View logs of a pod
<code>kubectl exec -it <pod-name> -- sh</code>	Access shell in a pod
<code>kubectl apply -f <file.yaml></code>	Apply configuration
<code>kubectl delete -f <file.yaml></code>	Delete configuration
<code>kubectl scale deployment <name> --replicas=3</code>	Scale deployment
<code>kubectl rollout status deployment/<name></code>	Check rollout status
<code>kubectl get all -n <namespace></code>	List all in a namespace



Deployment YAML (Example)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: nginx:latest
          ports:
            - containerPort: 80
```



Service Types

ClusterIP	Exposes service on a cluster-internal IP.
NodePort	Exposes service on each node's IP at a static port.
LoadBalancer	Exposes service externally using a cloud provider load balancer.
ExternalName	Maps service to an external DNS name.



Ingress Example

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
spec:
  rules:
    - host: myapp.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: myapp-service
                port:
                  number: 80
```



Best Practices



Use Namespaces to organize resources



Use Secrets for sensitive data



Use Deployments for updates & rollbacks



Set Resource Requests & Limits



Monitor your applications



Backup etcd regularly



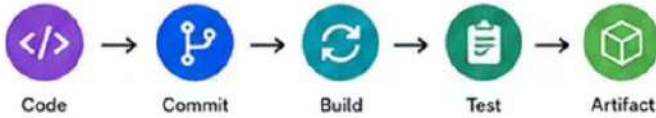
Keep Kubernetes updated

8. CI/CD



1. Continuous Integration (CI)

Developers frequently merge code changes into a shared repository. Each change is automatically built and tested.

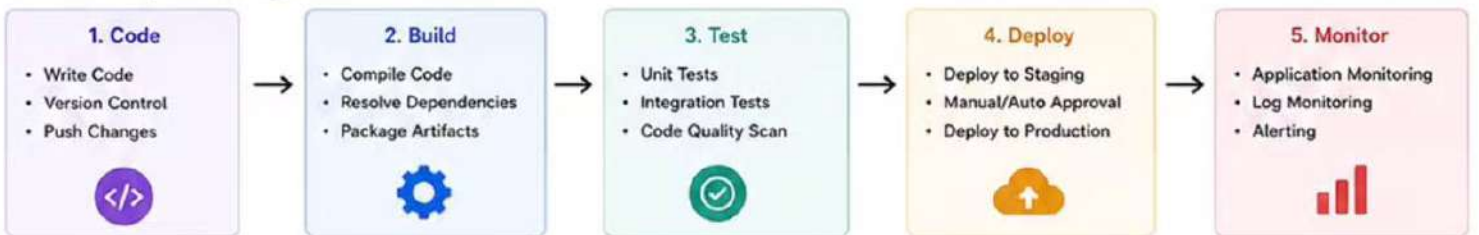


2. Continuous Delivery / Deployment (CD)

Code that passes all tests is automatically prepared for release (Delivery) or automatically released to production (Deployment).



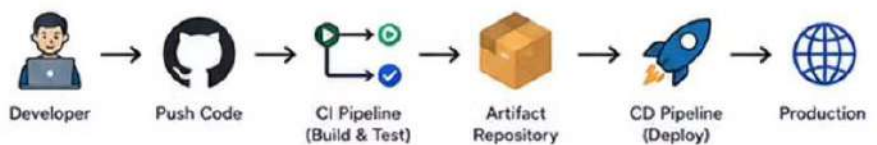
3. CI/CD Pipeline Stages



4. Popular CI/CD Tools

	GitHub Actions	Automate your workflow right in GitHub.
	Jenkins	Open-source automation server.
	GitLab CI/CD	Built-in CI/CD with GitLab.
	Azure DevOps	End-to-end DevOps solution from Microsoft.
	CircleCI	Automate your development pipeline.

5. Example CI/CD Flow



Continuous Delivery: Code is always in a deployable state.

Continuous Deployment: Code is automatically deployed to production.

6. Sample Tools Comparison

Tool	Type	Ease of Setup	Scalability	Integrations	Best For
GitHub Actions	CI/CD	★★★★☆	★★★★☆	GitHub, Docker, AWS, Kubernetes	GitHub users
Jenkins	CI/CD	★★★☆☆	★★★★★	Almost everything	Highly customizable pipelines
GitLab CI/CD	CI/CD	★★★★☆	★★★★☆	GitLab, Kubernetes, AWS, GCP	GitLab users
Azure DevOps	CI/CD	★★★★☆	★★★★☆	Microsoft, Azure, Docker, Kubernetes	Microsoft ecosystem
CircleCI	CI/CD	★★★★☆	★★★★☆	Docker, AWS, GCP, Kubernetes	Cloud-native teams

7. Common CI/CD Commands (Example - GitHub Actions)

```
name: CI/CD Pipeline
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '18'
      - run: npm install
      - run: npm test
      - run: npm run build
```



8. Best Practices

- ✓ Keep commits small and atomic.
- ✓ Automate build, test, and deployment.
- ✓ Run tests in parallel to save time.
- ✓ Use environment-specific configurations.
- ✓ Secure your secrets and credentials.
- ✓ Monitor pipelines and notify on failures.
- ✓ Keep your pipeline fast and reliable.



9. Benefits of CI/CD



9. Infrastructure as Code (IaC)

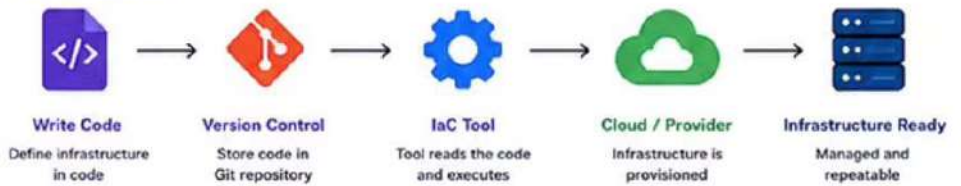


What is IaC?

Infrastructure as Code (IaC) is the practice of managing and provisioning infrastructure through machine-readable configuration files instead of manual processes.

- ✓ Consistent & Repeatable
- ✓ Version Controlled
- ✓ Automated & Scalable
- ✓ Faster & Reliable

How IaC Works



A. Terraform



Terraform is an IaC tool by HashiCorp used to provision and manage infrastructure across multiple cloud providers.

Key Concepts

- Provider: Defines the cloud platform (AWS, Azure, GCP, etc.)
- Resource: A component you want to create (VM, S3, VPC, etc.)
- Variables: Make code reusable and dynamic
- Modules: Reusable configuration
- State File: Tracks the current state of infrastructure

Terraform Workflow



Common Terraform Commands

Command	Description
terraform init	Initialize working directory
terraform plan	Preview changes
terraform apply	Apply changes
terraform destroy	Destroy infrastructure
terraform fmt	Format configuration files
terraform validate	Validate configuration files
terraform show	Show current state

Example: main.tf

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  tags = {
    Name = "Terraform-Instance"
  }
}
```

Terraform Benefits



B. Ansible



Ansible is an automation tool used for configuration management, application deployment, and task automation.

Key Concepts

- Inventory: List of hosts and groups
- Playbook: YAML file with tasks
- Module: Reusable unit of code
- Role: Organized collection of tasks, files, templates, handlers
- Idempotency: Ansible ensures the desired state is achieved

Ansible Architecture



Example: inventory.ini

```
[webservers]
web1 ansible_host=192.168.1.10
web2 ansible_host=192.168.1.11

[dbservers]
db1 ansible_host=192.168.1.20
```

Example: playbook.yml

```
- hosts: webservers
  become: yes
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
    - name: Start nginx
      service:
        name: nginx
        state: started
```

Common Ansible Commands

Command	Description
ansible all -m ping	Test connectivity to all hosts
ansible all -m setup	Gather facts about hosts
ansible-playbook playbook.yml	Run a playbook
ansible-inventory --list	List inventory
ansible-vault encrypt file.yml	Encrypt a file with vault
ansible-galaxy init role_name	Create a new role

C. Comparison: Terraform vs Ansible

Feature	Terraform	Ansible
Purpose	Provisioning Infrastructure	Configuration Management & Automation
Language	HCL (HashiCorp Configuration Language)	YAML
State Management	Yes (State File)	No (Agentless)
Idempotency	Yes	Yes
Focus	Infrastructure (Cloud Resources)	Configuration, Deployment, Orchestration
Example Use Cases	Create VPC, EC2, RDS, S3, IAM, etc.	Install packages, Configure servers, Deploy apps

D. Best Practices

- Store IaC code in Version Control (Git)
- Use meaningful module and resource names
- Follow least privilege access (IAM Roles)
- Use variables and outputs effectively
- Keep code DRY (Don't Repeat Yourself)
- Review changes with plan before apply
- Use remote backend for state (Terraform)
- Automate with CI/CD pipelines

E. Popular IaC Tools Ecosystem



PYTHON LIFE

10. Cloud Computing



What is Cloud Computing?

Cloud Computing is the delivery of computing services over the internet ("the cloud"), including servers, storage, databases, networking, software, analytics, and more, to offer faster innovation, flexible resources, and economies of scale.



Key Characteristics

- On-demand Self-service**
Provision resources automatically without human interaction.
- Broad Network Access**
Access services over the network from anywhere, anytime.
- Resource Pooling**
Provider's resources are pooled to serve multiple customers.
- Rapid Elasticity**
Scale resources up or down quickly as needed.
- Measured Service**
Pay only for what you use with metered billing.

Cloud Service Models

- IaaS** **Infrastructure as a Service**
Virtualized computing resources over the internet. You manage OS, storage, apps.
Examples: AWS EC2, Azure VM, Google Compute Engine
- PaaS** **Platform as a Service**
Platform for developing, running and managing applications without managing infrastructure.
Examples: Heroku, Google App Engine, Azure App Service
- SaaS** **Software as a Service**
Complete applications delivered over the internet. You just use the software.
Examples: Google Workspace, Microsoft 365, Salesforce

Deployment Models

- Public Cloud**
Services offered over the public internet by third-party providers.
- Private Cloud**
Cloud infrastructure used exclusively by a single organization.
- Hybrid Cloud**
Combination of public and private clouds with shared data & apps.
- Multi-Cloud**
Use of multiple cloud services from different providers.

Core Cloud Services

- Compute**
Virtual Machines, Containers, Serverless Functions
- Storage**
Object, Block, File Storage, Backup & Archival
- Database**
Relational, NoSQL, Managed Databases
- Networking**
VPC, Load Balancer, DNS, CDN, VPN, Firewall
- Analytics**
Big Data, Data Warehouse, Machine Learning
- Security & Identity**
IAM, Policies, Encryption, Key Management
- Management Tools**
Monitoring, Logging, Automation, Alerts
- DevOps Tools**
CI/CD, IaC, Configuration Management

Top Cloud Providers

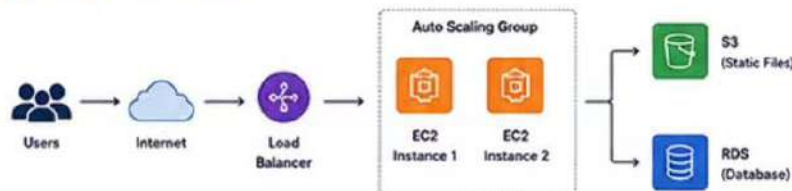
- aws** **Amazon Web Services (AWS)**
Most comprehensive cloud platform with global reach.
- A** **Microsoft Azure**
Enterprise-ready cloud with strong Microsoft ecosystem.
- GCP** **Google Cloud Platform (GCP)**
Data, analytics & ML strength with global infrastructure.
- ORACLE** **Oracle Cloud Infrastructure (OCI)**
High performance, secure and cost effective cloud solutions.

Example: Launch a Virtual Machine (AWS CLI)

```
# Configure AWS CLI
aws configure

# Launch an EC2 instance
aws ec2 run-instances \
  --image-id ami-0c55b159cbf1e1f0 \
  --count 1 \
  --instance-type t2.micro \
  --key-name my-key \
  --security-groups my-sg \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=MyEC2}]'
```

Example Architecture



Benefits of Cloud Computing

- ✓ **Lower Costs** – Pay-as-you-go pricing
- ✓ **Scalability** – Scale up or down in minutes
- ✓ **High Availability** – Built-in redundancy
- ✓ **Global Reach** – Deploy anywhere in the world
- ✓ **Security** – Advanced security & compliance
- ✓ **Innovation** – Focus on building, not managing infra

Cloud Security Best Practices

- ✓ Use IAM roles and least privilege access
- ✓ Enable Multi-Factor Authentication (MFA)
- ✓ Encrypt data at rest and in transit
- ✓ Regularly update and patch resources
- ✓ Monitor with logging and alerts
- ✓ Backup important data regularly

Common Cloud Pricing Models

- ✓ **On-Demand** – Pay for compute by the hour/second
- ✓ **Reserved** – Pay upfront for discounted rates
- ✓ **Spot / Preemptible** – Use spare capacity at lower cost
- ✓ **Pay-as-you-go** – Pay only for what you use
- ✓ **Free Tier** – Limited free services for learning/testing

Common Cloud Terms

- Region**
A geographical area with isolated locations.
- Availability Zone**
Physically separate data center in a region.
- VPC**
Virtual Private Cloud isolated network.
- SLA**
Service Level Agreement guarantees uptime.
- API**
Interface to manage cloud resources.
- IAM**
Identity and Access Management.

PYTHON LIFE

Key Takeaway

Cloud Computing delivers flexible, scalable and cost-effective solutions. Master the core concepts, services and best practices to build reliable and secure cloud-native applications.

